The Feedback Vertex Set Problem: Theory, Algorithms, and Applications

CSE472 Project Presentation - Group 06

2005090 – Tawkir Aziz Rahman
2005068 – Suman Hossain
2005074 – Dipanta Kumar Roy Nobo
2005091 – Waseem Mustak Zisan
2005104 – Hasin Arafat
2005109 – Noushin Tabassum Aoishy

Bangladesh University of Engineering and Technology
Department of CSE

April 6, 2026

Outline

1. Problem Definition and Visual Intuition
2. Complexity and NP-Completeness
3. Our Modifications
   - KMA: Kernelized Memetic Algorithm
   - DKMA: Dynamic Kernelized Memetic Algorithm
   - GNN-KMA Models
4. Results
   - Synthetic Dataset
   - PACE 2022 Benchmark
5. System Architecture
   - Results & Evaluation
6. Conclusions

Overview

Goal of This Section

- Understand what the Feedback Vertex Set (FVS) problem is
- Build intuition using cycles and forests
- Visualize how removing vertices eliminates cycles

What is a Cycle?

Definition

A **cycle** is a path that starts and ends at the same vertex without repeating edges.

- Cycles represent loops in a graph
- They create dependencies or redundancies

Key Idea

If a graph has no cycles, it is called a **forest**.

What is a Forest?

Definition

A **forest** is a graph with no cycles (i.e., a collection of trees).

- Trees are connected acyclic graphs
- Forest = one or multiple disconnected trees

Observation

Our goal in FVS is to transform any graph into a forest.

Feedback Vertex Set (FVS)

Definition

Given a graph $G = (V, E)$, a **Feedback Vertex Set** is a subset $S \subseteq V$ such that removing S makes the graph acyclic.

Objective

Find the **smallest possible** such set S.

- Remove vertices $\rightarrow$ eliminate cycles
- Minimize number of removed vertices

Problem Formulation

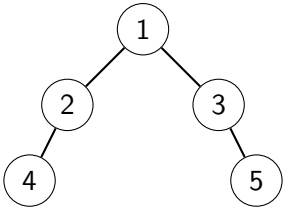
Decision Version

- Input: Graph $G = (V, E)$, integer k
- Question: Does there exist $S \subseteq V$ with $|S| \leq k$ such that $G - S$ is acyclic?

Optimization Version

Find a minimum-size Feedback Vertex Set.

Visual Example: Tree (No Cycles)

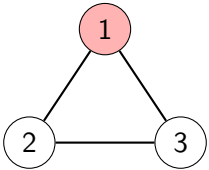


- No cycles exist
- Graph is already a forest

Result

FVS $= \emptyset$

Visual Example: Simple Cycle

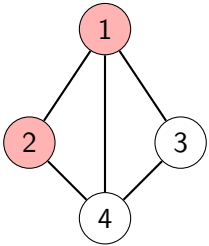


- This graph contains a cycle (triangle)
- Removing vertex 1 breaks the cycle

Result

FVS $= \{1\}$, size $= 1$

Visual Example: Multiple Cycles



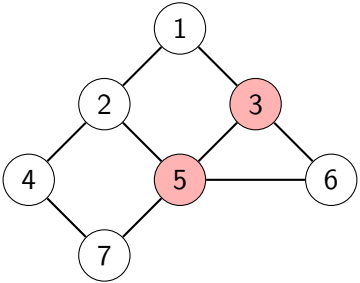
- Multiple overlapping cycles exist
- Need to remove more vertices

Result

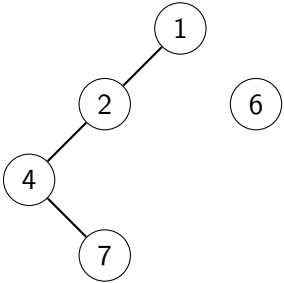
FVS $= \{1, 2\}$

Visual Example: Complex Graph → Forest

Before: Cycles



After: Forest



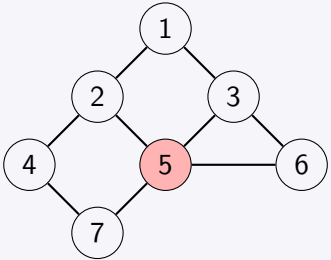
- **Before:** Multiple cycles exist, highlighted red vertices (3,5) are part of FVS.
- **After:** Removing FVS nodes breaks all cycles → Graph becomes a acyclic.

Final Verdict

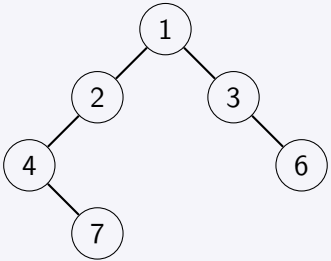
FVS $= \{3, 5\}$ → A set of vertices to remove all cycles.

Visual Example: Complex Graph → Minimal FVS → Forest

Original Graph with Cycles



Resulting Forest (Remove 5)



- Minimal Feedback Vertex Set: $\{5\}$
- Removing vertex 5 breaks all cycles → graph becomes a forest

Key Insight

Even if multiple vertices can break cycles, the **minimal FVS** is the set with the smallest size.

Real-World Motivation

- **Deadlocks in Operating Systems:** Cycles in resource allocation graph $\rightarrow$ remove critical process
- **Dependency Resolution:** Packages/modules with circular dependencies $\rightarrow$ remove one to break cycle
- **Circuit Design / Feedback Loops:** Loops in circuits $\rightarrow$ remove components to stabilize

Insight

FVS is a tool to identify minimal critical nodes whose removal resolves all cycles.

Key Observations

- Every cycle must contain at least one removed vertex
- Removing fewer vertices is always better
- Problem becomes harder as graph becomes denser

Core Insight

FVS is about selecting a minimum set of vertices that "hit" all cycles.

Complexity Overview

Goal of This Section

- Analyze the **computational complexity** of FVS
- Understand decision vs optimization versions
- Prove NP-Completeness using reductions

Decision vs Optimization

Decision Version

Given G, k: Does there exist an FVS of size $\leq k$?

Fact

Decision version is **NP-Complete**

Optimization Version

Find the minimum-size FVS

Fact

Optimization version is **NP-Hard**

Main Claim

Theorem

The decision version of Feedback Vertex Set is **NP-Complete**.

Proof Strategy

1. Show FVS $\in$ NP
2. Reduce Vertex Cover $\leq_p$ FVS

Step 1: FVS is in NP

Key Idea (NP Definition)

A problem is in NP if a solution can be **verified in polynomial time**.

- Given a set S
- Remove S from graph
- Check if graph has cycles (DFS)

Time Complexity

$O(V + E)$

Conclusion

FVS $\in$ NP (verification is polynomial)

Step 2: Reduction (VC $\rightarrow$ FVS)

Known Fact

Vertex Cover is NP-Complete

Goal

Transform VC instance into FVS instance

- Each edge becomes a cycle
- FVS must break all cycles

Step 2: Reduction (VC $\rightarrow$ FVS)

Vertex Cover (VC) Problem

Find smallest $S \subseteq V$ such that every edge has at least one endpoint in S.

Construction: Given VC instance (G, k)

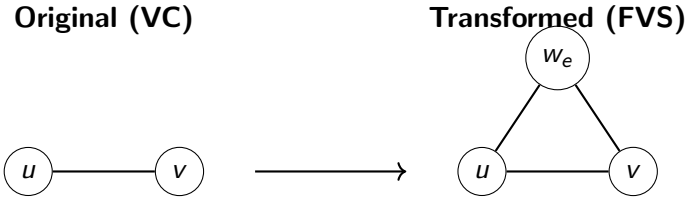
1. For each edge $e = (u, v) \in E(G)$:
   - Add new vertex w_e
   - Form triangle: edges (u, v), (u, w_e), (v, w_e)
2. Keep same parameter $k' = k$

Theorem

G has VC of size $\leq k \Leftrightarrow G'$ has FVS of size $\leq k$

Time: $O(|E|)$ (polynomial)

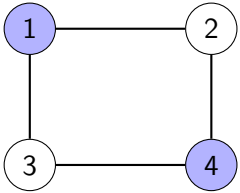
Reduction Construction (Graph)



- Each edge becomes a triangle
- Triangle = cycle → must be broken

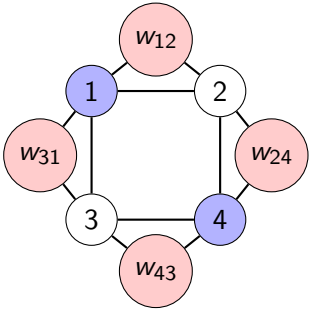
VC $\rightarrow$ FVS: Full Graph Illustration

Original Graph



VC $= \{1, 4\}$

Transformed Graph



Each triangle = one edge

Key Insight

Breaking all triangles = covering all edges

Correctness of Reduction

Claim

G has a VC of size $\leq k \iff G'$ has an FVS of size $\leq k$

Forward:

- VC selects endpoints $\rightarrow$ every triangle broken

Backward:

- FVS must hit each triangle
- Implies every edge is covered

Polynomial Time Reduction

- Add one vertex per edge
- Linear size increase

Conclusion

Reduction runs in polynomial time

Reverse Reduction (FVS $\rightarrow$ VC)

Reverse Direction

Shows Vertex Cover and FVS are polynomially equivalent

Construction: Given FVS instance (G, k)

1. Replace each vertex $v \in V$ with triangle $\{v_1, v_2, v_3\}$
2. For each edge $(u, v) \in E$, connect triangles with cross edges
3. Set $k' = 2|V| + k$

Theorem

G has FVS of size $\leq k \Leftrightarrow G'$ has VC of size $\leq 2|V| + k$

Key Insight:

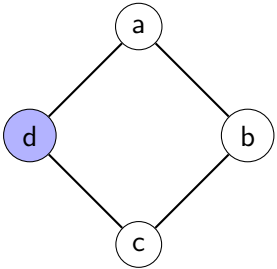
- Each triangle needs ≥ 2 vertices in VC
- Triangles with 3 vertices in VC, $\leftrightarrow$ vertices in FVS

FVS→ VC: Visual Example

Idea

Show structural similarity between problems

Original Graph



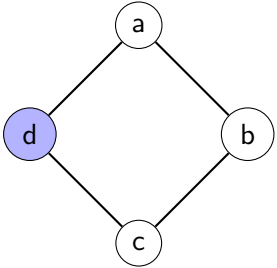
Construction

- Each vertex $\rightarrow$ triangle
- Selecting all 3, $\leftrightarrow$ remove vertex
- Selecting 2, $\leftrightarrow$ keep vertex

FVS $= \{d\}$

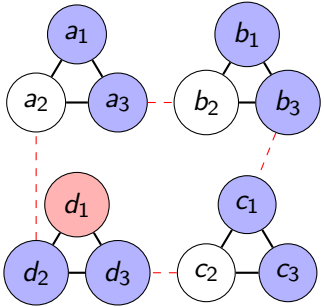
FVS→ VC: Visual Example

Original Graph G:



FVS: $\{d\}$ (size=1)
Removing d breaks the 4-cycle

Transformed Graph G':



Dashed = original edges , Each triangle = one vertex in G

Vertex Cover in G':(with d1) is a FVS in G'

Final Conclusion

We Have Proven

- FVS $\in$ NP
- Vertex Cover $\leq_p$ FVS
- VC is NP-Complete

Final Verdict

Feedback Vertex Set (Decision) is NP-Complete

Implication

- Optimization version is NP-Hard
- No known polynomial-time exact algorithm

Algorithm Selection Criteria

Essential Criteria

1. **Implementability**
   - Feasible in 6-week timeframe
   - Well-documented in literature
   - Reasonable debugging complexity
2. **Research Potential**
   - Novel comparisons possible
   - Room for improvements
   - Publishable results
3. **Complementary Nature**
   - Different problem sizes
   - Different paradigms
   - Interesting comparisons

Additional Criteria

4. **Practical Relevance**
   - Real-world applications
   - Industry interest
   - Addresses practical needs
5. **Educational Value**
   - Learn important techniques
   - Transferable skills
   - Deep algorithmic insights

Goal

Select algorithms that maximize research impact while ensuring feasibility

Selected Algorithms for Implementation

Three-Algorithm Approach (Maximum Research Impact)

We will implement three complementary algorithms covering the entire solution spectrum:

1. **Iterative Compression** (Exact FPT Algorithm)
   - Guarantees optimal solution for small-medium k
   - Time: $O(5^k \cdot kn^2)$ — FPT breakthrough (2004)
   - Research: Hybrid approaches, parallelization, adaptive branching
2. **Kernelization + Bounded Search Tree** (Enhanced Exact)
   - Preprocessing with reduction rules + exact search
   - Kernel reduction: 50-90% for many graph types
   - Research: New reduction rules, smart branching strategies
3. **Memetic Algorithm** (Advanced GA + Local Search)
   - Handles large instances (n > 1000)
   - Combines population evolution with local optimization
   - Research: Problem-specific operators, ML integration, hybrid exact-metaheuristic

Algorithm 1: Iterative Compression (IC)

Core Concept

Given FVS of size $k + 1$, compress it to size k through smart enumeration

Theoretical Significance:

- Most important FPT technique
- Published in STOC/FOCS
- Demonstrates advanced understanding

Implementation:

- Complexity: Medium
- Expected range: $k \leq 30$
- Validation on small instances

Research Directions:

1. **Hybrid Kernelization + IC**
   - Apply reduction rules before IC
   - Potential 2-5x speedup
2. **Adaptive Branching**
   - Learn which vertices to branch first
   - Use graph structure metrics
3. **Parallelization**
   - Independent branches in parallel
   - Multi-core speedup potential
4. **Improved Compression**
   - Smart partition enumeration
   - Structure-based pruning

Iterative Compression for FVS

Idea: Build solution incrementally and compress when too large.

Algorithm

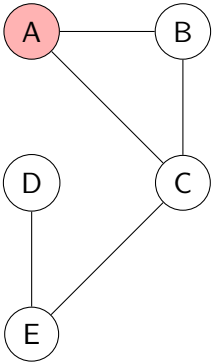
```
Order vertices v_1, ..., v_n
F ← {v_1}
for i = 2 to n do
    F ← F ∪ {v_i}
    if |F| = k + 1 then
        for all partitions (F_1, F_2) of F do
            find Y ⊆ V \ F_2
            if F_1 ∪ Y is an FVS then
                update F and break
            end if
        end for
        if no update performed then
            return NO
        end if
    end if
end for
return F if |F| ≤ k
```

Key Insight: Compress size $k+1$ to k

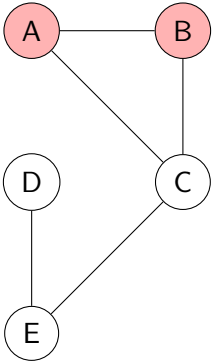
Time: $O(5^k \cdot n^2)$

Simulation

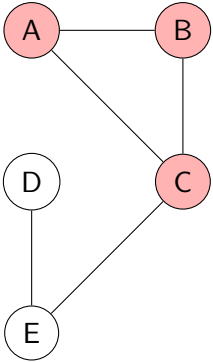
Simulation of FVS (k=2) on 5 vertices
Step 1: $F = \{A\}$



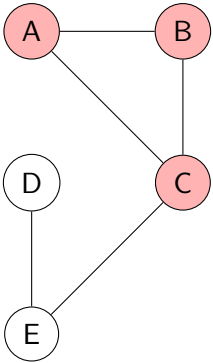
Step 2: $F = \{A, B\}$



Step 3: $F = \{A, B, C\}$ (overflow)

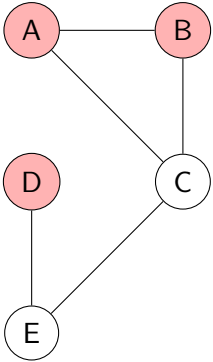


Step 3: $F = \{A, B, C\}$ (overflow)

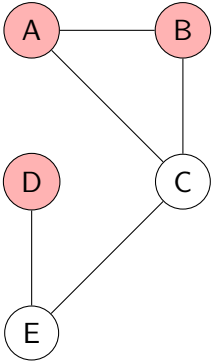


Fix: Remove C $\Rightarrow F = \{A, B\}$

Step 4: Add D (Overflow)

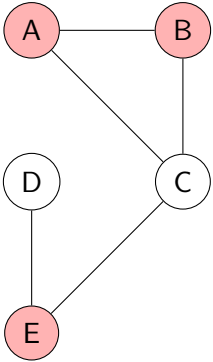


Step 4: Add D (Overflow)

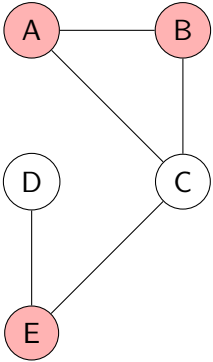


Fix: Remove D $\Rightarrow F = \{A, B\}$

Step 5: Add E (Overflow)

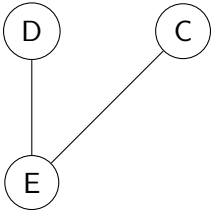


Step 5: Add E (Overflow)



Fix: Remove E $\Rightarrow F = \{A, B\}$

Final Answer



$F = \{A, B\}$

Algorithm 2: Kernelization + Bounded Search Tree

Core Concept

Reduce graph size via preprocessing rules, then apply exact search on smaller kernel

Reduction Rules:

- Degree 0, 1, 2 vertices
- Self-loops, duplicate edges
- Crown decomposition
- Flower rule

Search Strategy:

- Smart vertex selection
- Bounded depth: k
- Branching factor: ≤ 4

Research Directions:

1. **New Reduction Rules**
   - Graph-specific rules
   - Theoretical contribution
2. **Rule Application Order**
   - Optimize ordering
   - ML-based prediction
3. **Kernelization Bounds**
   - Tighter bounds for special classes
   - Experimental validation
4. **Smart Branching**
   - Centrality-based selection
   - ML-guided search

Kernelization + Bounded Search Tree

Idea: Reduce the graph, then branch on remaining cycles.

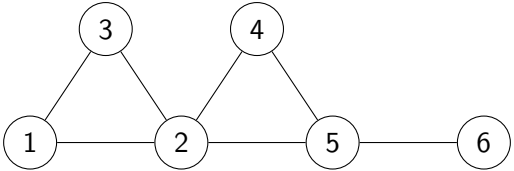
Algorithm

```
Apply reduction rules:
    remove degree ≤ 1 vertices
    contract degree-2 vertices
    include self-loop vertices
Obtain kernel (G', k')
function Search(G', k')
if G' is acyclic then
    return solution
end if
if k' < 0 then
    return NO
end if
find a cycle C
for each v ∈ C do
    recurse on (G' − v, k' − 1)
end for
```

Key Insight: Small kernel $\Rightarrow$ small search tree

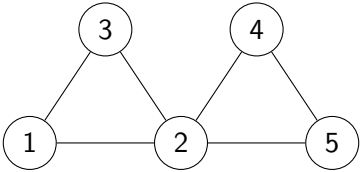
Time: $O(4^k + n^2)$

Step 1: Original Graph

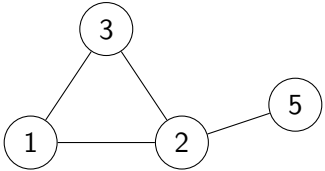


Step 2: Remove degree ≤ 1 vertices

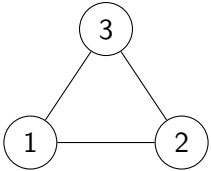
Vertex 6 removed



Step 3: Contract degree-2 vertex (4)

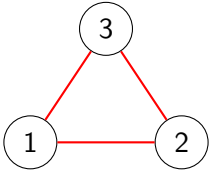


Step 4: Remove degree ≤ 1 vertex (5)

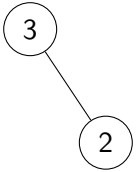


Step 5: Kernel Graph (G')

Remaining cycle

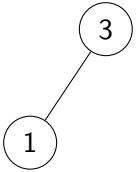


Step 6: Branch - Remove vertex 1



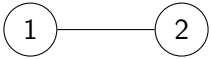
Graph becomes acyclic

Step 7: Branch - Remove vertex 2



Graph becomes acyclic

Step 8: Branch - Remove vertex 3



Graph becomes acyclic

Algorithm 3: Memetic Algorithm (GA + Local Search)

Core Concept

Population evolution + local optimization for large-scale instances

Components:

- Smart initialization (greedy + random)
- FVS-aware crossover operators
- Centrality-based mutation
- Problem-specific local search

Practical Relevance:

- Industry standard
- Handles $n > 1000$ vertices
- Near-optimal (5-10% gap)

Research Directions:

1. **Cycle-Preserving Crossover**
   - Preserve cycle-breaking structures
   - Novel contribution
2. **Hybrid Exact-Metaheuristic**
   - Use IC for small components
   - GA for large core
3. **ML Integration**
   - Predict promising vertices
   - Learn operator selection
4. **Multi-Objective**
   - Minimize size + maximize robustness
   - Pareto frontier

Memetic Algorithm for FVS

Idea: Evolutionary search combined with local optimization.

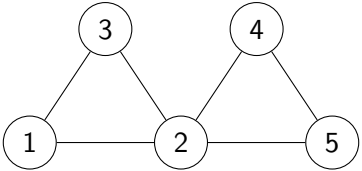
Algorithm

```
Encode solution as a permutation of vertices
Decode by removing vertices until graph is acyclic
Initialize population (greedy + random)
for each generation do
    Evaluate fitness: −|FVS|
    Select parents (tournament)
    Crossover (cycle-aware)
    Mutation (swap cycle-heavy vertices)
    Local search (hill-climbing)
end for
return best solution found
```

Key Insight: Exploration + refinement

Type: Heuristic (near-optimal)

Step 1: Original Graph

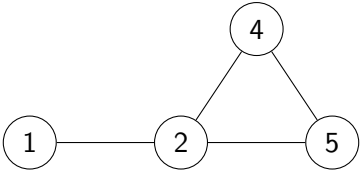


Step 2: Encode Solution

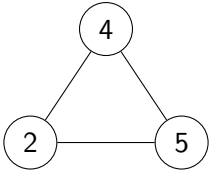
Example permutation:

$$[3, 1, 5, 2, 4]$$

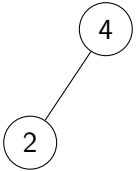
Step 3: Remove vertex 3



Step 4: Remove vertex 1



Step 5: Remove vertex 5



Graph is now acyclic $\rightarrow$ Stop decoding

Step 6: Fitness Evaluation

$$\text{Removed vertices} = \{3,1,5\}$$

$$\text{Fitness} = -|FVS| = -3$$

Step 7: Selection (Tournament)

Select best solutions from population

Example:

$$[3, 1, 5, 2, 4], \quad [2, 4, 1, 5, 3]$$

Step 8: Crossover

Parent 1: [3,1,5,2,4]
Parent 2: [2,4,1,5,3]

Child:

[3,4,1,2,5]

Step 9: Mutation

Before:

$$[3, 4, 1, 2, 5]$$

After swap:

$$[3,1,4,2,5]$$

Step 10: Local Search

Try improving solution:

Remove fewer vertices if possible

Example improvement:

$$\text{FVS size: } 3 \rightarrow 2$$

Step 11: Best Solution

Best FVS found:

$$\{3,2\}$$

Graph becomes acyclic

Our Modifications

KMA DKMA GNN-KMA

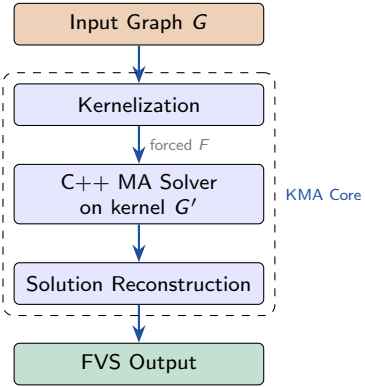
KMA — Motivation and High-Level Design

Why not pure Memetic Algorithm?

- A plain MA [7] operates on the full graph
- Reduction rules can *safely* eliminate many vertices before any search [3]
- This shrinks the search space, letting MA focus on hard subgraphs

KMA Pipeline

1. **Kernelization** — apply reduction rules, record forced vertices
2. **MA on kernel** — C++ evolutionary search on reduced graph
3. **Reconstruction** — map kernel solution to original indices, union with forced vertices



KMA — Kernelization Rules

Undirected Kernelization Rules

1. **Self-loop:** vertex with self-loop $\Rightarrow$ force into FVS
2. **Degree-0/1:** acyclic leaf $\Rightarrow$ safely delete
3. **Degree-2 bypass:** if $\deg(v) = 2$ with neighbors a, b and $\{a, b\} \notin E$: delete v, add edge (a, b)

Rules applied **iteratively** until quiescence.

Theoretical basis: $O(k^2)$ kernel [3].

Directed Kernelization Rules

1. **Self-loop:** forced into DFVS
2. **Source/Sink:** in-deg $= 0$ or out-deg $= 0 \Rightarrow$ delete (not on any directed cycle)
3. **Degree-1 bypass:** in-deg or out-deg $= 1 \Rightarrow$ shortcut edges, delete vertex
4. **SCC reduction:** retain only vertices in nontrivial SCCs (size > 1 or self-loop)

Directed kernel theory: [6].

Modification over baseline MA: The addition of kernelization (especially the SCC-based directed reduction) is a key novelty. SCC reduction alone can eliminate **60–90%** of vertices on sparse real-world directed graphs before any search begins.

KMA — Memetic Algorithm Core

C++ MA Solver (evolutionary search on kernel):

- Initialise population of size P randomly
- Each individual: binary vector over kernel vertices
- Each generation:
  1. **Crossover** (two parents $\rightarrow$ offspring)
  2. **Local mutation** (flip, swap)
  3. **Repair**: restore FVS validity via greedy cycle cover
  4. **Selection**: tournament
- Early-stop if no improvement for T generations
- Hard wall-clock timeout enforced inside C++

Parameters

Param	Flag	Default
Population	`--pop`	20
Max generations	`--gens`	100
Timeout	`--timeout`	600s
Early-stop	`--earlystop`	30

Complexity Note

Kernelization: $O(n + m)$ per pass
MA on kernel G': heuristic,
$O(P \cdot \text{gen} \cdot |V(G')|)$
Full pipeline: practical for
$|V| \leq 10^4$ within timeout

DKMA - Dynamic Kernelized Memetic Algorithm — Core Idea

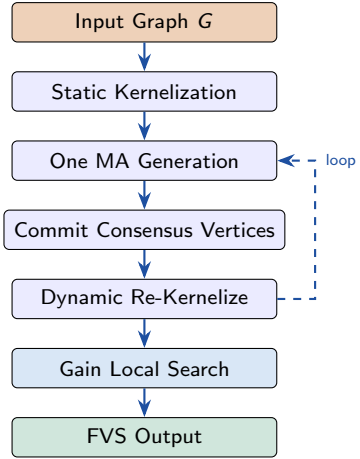
Limitation of static kernelization (KMA):

- Kernelization is applied *once* before MA
- Partial solutions during MA may *unlock* new reductions — these are never exploited

DKMA — Dynamic interleaving [8, 9]:

1. Evolve population for one MA generation
2. **Commit** vertices appearing in $\geq \tau$ fraction of individuals
3. Force committed vertices into the solution
4. **Re-kernelize** residual graph
5. Continue MA on the smaller kernel

This **opening-up effect** progressively shrinks the kernel as partial agreement is reached.



DKMA — Key Innovations & Helpers

SHORTONE Directed Contractions

Inspired by the Mount-Doom PACE 2022 solver [10], a directed contraction pass (`_apply_shortone_rule`) is applied before the MA loop on directed graphs.

Contracts degree-1 in/out vertices with special cycle-preserving rewiring, enabling additional SCC reductions.

Gain Local Search (post-loop)

`_gain_local_search` applies **1-1** and **2-1** improvement swaps after the MA loop:

- **1-1 swap**: remove one FVS vertex, add another if acyclicity is preserved and size decreases
- **2-1 swap**: replace two FVS vertices with one

Topological Diversification

Every 5 generations, `_diversify_with_topological_ordering` is called:

- Directed: randomised Kahn's algorithm tie-breaking
- Undirected: random spanning-tree cycle-breaking insertion

Prevents premature convergence [7].

Commit Threshold & Cap

`_commit_vertices_from_population`:

- Commit if vertex appears in $\geq \tau$ of population
- Cap at top 30% by frequency if over-committing
- Default $\tau = 0.8$

Based on DFVS IPEC 2022 approach [8].

DKMA — Correctness & Comparison with KMA

Correctness Safeguards

- Deduplicated final solution indices
- All forced vertices (static + dynamic) included
- Final **acyclicity verification** on original graph: directed $\Rightarrow$ iterative DFS back-edge detection;
  undirected $\Rightarrow$ union-find cycle check
- **Fallback to KMA** if validation fails

KMA vs DKMA — Key Differences

Feature	KMA	DKMA
Kernelization	Once	Per loop
Commitment	None	Consensus
SHORTONE pass	No	Yes (directed)
Topological div.	No	Every 5 gen.
Local search	No	Gain 1-1/2-1
Acyclicity check	End	End + fallback

References: DKMA design is inspired by the PACE 2022 FVS track winner strategies [8] (Grossmann, Heuer, Schulz, Strash, IPEC 2022) and the ESA 2022 DFVS analysis [9] (Figiel, Froese, Nichterlein, Niedermeier). The Mount-Doom SHORTONE rule is from Langedal et al. [10].

GNN-KMA — Hybrid Coupling Strategy

Motivation:

- GNN learns vertex-level patterns from solved instances
- Predicts probability $p(v)$ of belonging to FVS
- High-confidence predictions **hard-fix** vertices before KMA, reducing the search problem

Precision-first soft-hint coupling:

1. Kernelize graph $G \to G'$
2. Run GNN on G', get $p(v)$ for each v
3. Hard-fix only vertices with $p(v) \geq \theta$
4. Cap hard-fix at **8%** of kernel size
5. Run KMA on residual kernel
6. Reconstruct: forced $\cup$ GNN-fixed $\cup$ KMA output

Default: $\theta = 0.65$, min-fix $= 0.5\%$, max-fix $= 8\%$ of kernel

Why Precision-First?

False positives in hard-fixing are **catastrophic**: a wrongly fixed vertex inflates the FVS size permanently within that branch. Capping at 8% and requiring $\theta \geq 0.65$ ensures the GNN only commits when highly confident.

If no vertex exceeds θ, the system **falls back to pure KMA** automatically.

Can you do a GNN-DKMA variant? Same coupling, but the residual kernel is handed to DKMA instead of KMA, enabling dynamic re-kernelization atop GNN guidance.

GNN-KMA v1 — Baseline Architecture

Feature Engineering (3 channels)

Undirected:

- Normalised degree $d(v)/d_{\max}$
- Clustering coefficient $c(v)$ [18]
- Normalised log-degree

Directed:

- Normalised in-degree
- Normalised out-degree
- Normalised min(in, out)-degree

Design Choice

Clustering coefficient [18] captures local cycle density — vertices with high $c(v)$ are strong FVS candidates. This is a well-established heuristic from the small-world network literature.

Model Architecture

UndirectedFVSNet:

- 3-layer **GraphSAGE** message passing [11]
- BatchNorm + Dropout after each layer
- MLP classification head
- 2-class `log-softmax` output

DirectedFVSNet:

- Custom directed aggregation layer (separate in / out / self terms)
- 3 layers + BN + Dropout + MLP
- 2-class `log-softmax` output

Training Objective

Weighted NLL loss + cosine LR schedule.
Checkpoint by **validation F1** score.

GraphSAGE [11]: Hamilton et al., NeurIPS 2017.

GNN-KMA v2 — Richer Structural Features

Feature Engineering (11 channels)

1. Degree-like channels (in/out/min or undirected)
2. **RWSE** — Random Walk Structural Encoding [13]
   short-range return probabilities at steps $\{2, 3, 4\}$
3. **Motif counts**: triangle count, 4-cycle participation, 4-clique style counts [14]
4. **k-core number** $\kappa(v)$ [15]

Architecture & Training

UndirectedFVSNetV2 / DirectedFVSNetV2:

- Same GraphSAGE-style backbone as v1
- Upgraded input: `in_channels=11`
- Richer representation $\Rightarrow$ stronger structural discrimination

Training: same NLL + cosine schedule; checkpoint by validation F1.

Why These Features?

RWSE [13]: captures short-range cycle return probability — directly relevant to FVS membership.

k-core [15]: vertices deep in dense cores are more likely to be in any FVS.

Motif counts [14]: triangle/4-cycle participation indicates cycle exposure.

GNN-KMA v3 — Attention-Based Precision Model

Feature Engineering (16 channels)

- Extended **RWSE** horizon: steps $\{2, 3, 4, 6, 8, 12, 16\}$ [13]
- **SCC-aware channels** for directed cycle structure (SCC size, SCC membership depth)
- Cycle-score interaction features
- Directional ratio features (in/(in + out))
- Reduced reliance on expensive motif channels

SCC Channels (Directed)

SCC decomposition [16] provides structural depth: vertices in large SCCs are more critical for DFVS. SCC-size and SCC-depth channels directly encode this.

DirectedFVSNetV3 Architecture

- Input projection layer
- **5 residual GATv2 blocks** [12]
- Separate **forward** and **reverse** message paths
- Per-layer fusion projections
- **Global readout context** concatenated to node embedding
- Single-logit output per node (sigmoid)

v3 Training Innovations

- **AsymmetricFVSLoss**: penalises false positives more strongly (precision-oriented)
- Warmup + cosine LR + **gradient clipping**
- Checkpoint metric: **topk-precision@8%** (matches hard-fix cap)
- 200 epochs (vs 100 for v1/v2)

GNN-KMA — Feature & Architecture Evolution Summary

Aspect	**v1**	**v2**	**v3**
Feature channels	3	11	16
Degree features	✓	✓	✓
Clustering coeff. [18]	✓	–	–
RWSE [13]	–	steps $\{2, 3, 4\}$	steps $\{2..16\}$
Motif counts [14]	–	✓	reduced
k-core [15]	–	✓	–
SCC channels [16]	–	–	✓
Backbone	GraphSAGE [11]	GraphSAGE	GATv2 [12]
Depth	3 layers	3 layers	5 residual blocks
Bidirectional	No	No	Yes (fwd+rev)
Global readout	No	No	Yes
Loss	NLL	NLL	AsymmetricFVSLoss
Checkpoint metric	Val F1	Val F1	topk-prec@8%
Epochs	100	100	200

Dataset — Design Goals & Generation Pipeline

Five design goals:

1. **Controlled distribution** — fixed category proportions
2. **Track-aware difficulty** — separate exact & heuristic tracks
3. **Reproducibility** — deterministic seeding throughout
4. **Restartability** — safe reuse of existing files
5. **Benchmark/train alignment** — PT datasets mirror TXT corpus structure

Three-stage orchestration:

1. `download_real_world.py`
   NetworkX, PyG, OGB, SNAP sources
2. `generate_synthetic.py`
   Barabási–Albert, Watts–Strogatz, ER, grids, trees, DAGs
3. `dataset_gen.py`
   TXT → PyG Data objects with solver labels

Directory Contract

```
data/synthetic/
{undirected|directed}/
{exact_track|heuristic_track}/
{category}/
*.txt
```

Track Size Intent

Track	Node range
exact_track	≈ 10–35 nodes
heuristic_track	≈ 100–5000 nodes

Default: 100K undirected + 100K directed
exact ratio = 0.5, seed = 1337

Dataset — Undirected Distribution

Undirected Categories (20% each)

Category	Weight	Graph Model
`real_world`	20%	NetworkX/PyG/OGB/SNAP
`scale_free`	20%	Barabási–Albert [17]
`small_world`	20%	Watts–Strogatz [18]
`random_er`	20%	Erdős–Rényi [19]
`grids_trees`	20%	Grid + Random tree

Note: The undirected dataset balances real-world graphs, scale-free networks, small-world models, random Erdős–Rényi graphs, and structured grids/trees.

Dataset — Directed Distribution

Directed Categories

Category	Weight
`real_world_ego`	30%
`scale_free`	20%
`random_er`	20%
`directed_grids`	15%
`dags`	15%

Note: Directed instances include ego-networks, scale-free digraphs, random directed graphs, grid-based graphs, and acyclic DAG structures.

Dataset — Real-World Share & Labels

Real-world share:
Undirected: 20% real-world graphs
Directed: 30% ego-network graphs

Sources: NetworkX built-ins, PyG datasets, OGB node property datasets, SNAP archives.

Cached under `.cache/real_graphs`.

Label Sources

exact_track: IC solver labels
(high-fidelity, near-optimal)

heuristic_track: KMA labels
(practical, large-instance behavior)

Dataset — Why We Used This Approach

Diversity

- **5 undirected families**: covers scale-free hubs, small-world clusters, random structure, regular grids/trees, and real topology
- **5 directed families**: adds ego networks and DAGs
- Both undirected and directed instances
- Two difficulty regimes (exact / heuristic)

Quality Guarantees

- Each label validated via acyclicity check
- Subprocess timeout isolation for all solver calls
- CSV status ledger tracks completed / timeout / invalid outcomes
- Largest connected component extracted
- Compact integer relabelling applied
- No self-loops in edge lists

Scale

100K total undirected + 100K directed
(20K version also available on Kaggle)

Largest publicly available FVS dataset with ground-truth labels to our knowledge.

Kaggle (100K):
`kaggle.com/.../fvs-synthetic-dataset-100k`

Kaggle (20K):
`kaggle.com/.../fvs-synthetic-dataset-20k`

PACE 2022 Dataset — Real-World Benchmark Testing

What is PACE 2022?

The **Parameterized Algorithms and Computational Experiments** (PACE) Challenge 2022 [8] focused on the **Feedback Vertex Set** problem.

It provided:

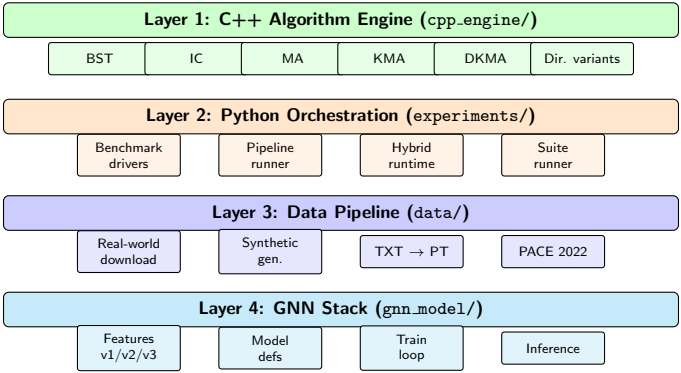
- 200 public undirected test instances
- Curated from real-world and synthetic sources
- Difficulty ranges from small exact to large heuristic
- Official solution checker for validation
- World-class solver implementations as baselines

Usage in This Project

- PACE 2022 instances stored under `data/pace2022/`
- **Protected from cleanup** — never deleted by `setup_benchmark_inputs.py`
- Benchmarked via `pacerun.sh` and `paceresults/`
- Used to evaluate **heuristic solvers** (KMA, DKMA, GNN-KMA v1/v2/v3) against real competition-grade instances
- PACE winner results in `pace2022_winner.csv` used as comparison baseline

Why PACE 2022? Our synthetic dataset covers diverse graph families but may not capture the nuances of competition-curated real-world instances. PACE 2022 instances serve as a *held-out real-world stress test* for heuristic solvers, enabling direct comparison with state-of-the-art PACE competitors [8].

End-to-End System Architecture



Evaluation Framework

Benchmark Output Format

Each benchmark run produces a CSV with:

- File identity & graph metadata
- Algorithm / family / track tags
- `fvs_size`: solution size
- `runtime`: wall-clock seconds
- `valid`: acyclicity-verified boolean
- `status`: completed / timeout / error

Benchmark Tracks

exact_track: compared against IC/BST optimal
heuristic_track: compared against MA baseline
PACE 2022: compared against official winner

Algorithm Comparison Matrix

Algorithm	Exact	Heuristic
BST	✓	–
IC	✓	–
MA	–	✓
KMA	–	✓
DKMA	–	✓
GNN-KMA v1	–	✓
GNN-KMA v2	–	✓
GNN-KMA v3	–	✓
GNN-DKMA	–	✓

Resume semantics: all benchmark pipelines support deterministic restart — interrupted runs continue from last completed file.

Summary & Contributions

Algorithm Contributions

- **KMA**: first-class kernelization (SCC reduction for directed; degree-2 bypass for undirected) integrated with C++ memetic search
- **DKMA**: dynamic re-kernelization loop, SHORTONE contractions [10], topological diversification, gain local search (1-1 / 2-1 swaps)
- **GNN-KMA v1/v2/v3**: progressive feature and architecture improvements (GraphSAGE $\rightarrow$ GATv2 [12], 3 $\rightarrow$ 16 channels, NLL $\rightarrow$ asymmetric precision loss)

Dataset Contributions

- 100K undirected + 100K directed labelled FVS instances
- 5 graph families, 2 difficulty tracks, real+synthetic mix
- Solver-verified ground-truth labels (IC for exact, KMA for heuristic)
- Publicly released on Kaggle (100K and 20K versions)
- PACE 2022 integration for real-world benchmarking

GitHub:

References I

- R.M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, 1972, pp. 85–103.
- A. Becker and D. Geiger, "Optimization of Pearl's method of conditioning and greedy-like approximation algorithms for the weighted maximum acyclic subgraph problem," *Artificial Intelligence*, vol. 105, pp. 1–2, 1996.
- S. Thomassé, "A $4k^2$ kernel for feedback vertex set," *ACM Trans. Algorithms*, vol. 6, no. 2, 2010.
- B. Reed, K. Smith, and A. Vetta, "Finding odd cycle transversals," *Operations Research Letters*, vol. 32, no. 4, 2004.
- M. Cygan *et al.*, *Parameterized Algorithms*. Springer, 2015.
- J. Chen, Y. Liu, S. Lu, B. O'Sullivan, and I. Razgon, "A fixed-parameter algorithm for the directed feedback vertex set problem," *J. ACM*, vol. 55, no. 5, 2008.
- T. Lust and J. Teghem, "The multiobjective multidimensional knapsack problem: a survey and a new approach," *Int. Trans. Oper. Res.*, 2010. (Memetic algorithm framework reference.)
- G. Grossmann, M. Heuer, P. Schulz, and D. Strash, "Targeted solution methods for the minimum feedback vertex set problem," in *Proc. 17th IPEC*, LIPIcs vol. 249, 2022, doi:10.4230/LIPIcs.IPEC.2022.26.
- A. Figiel, V. Froese, A. Nichterlein, and R. Niedermeier, "Improved parameterized algorithms for the directed feedback vertex set problem," in *Proc. ESA 2022*, LIPIcs, 2022, doi:10.4230/LIPIcs.ESA.2022.53.

References II

- K. Langedal, J. Langguth, F. Manne, and D. Strash, "Efficient minimum weight vertex cover heuristics using graph neural networks" (Mount-Doom FVS solver), Zenodo, 2022, doi:10.5281/zenodo.6630611.
- W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs" (GraphSAGE), in *NeurIPS*, 2017.
- S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" (GATv2), in *ICLR*, 2022.
- V.P. Dwivedi *et al.*, "Benchmarking graph neural networks," *JMLR*, vol. 24, no. 43, 2023. (RWSE/positional encoding reference.)
- N.K. Ahmed, J. Neville, R.A. Rossi, and N. Duffield, "Efficient graphlet counting for large networks," in *ICDM*, 2015.
- V. Batagelj and M. Zaversnik, "An $O(m)$ algorithm for cores decomposition of networks," *arXiv:cs/0310049*, 2003.
- R.E. Tarjan, "Depth-first search and linear graph algorithms," *SIAM J. Comput.*, vol. 1, no. 2, 1972.
- A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, 1999.
- D.J. Watts and S.H. Strogatz, "Collective dynamics of 'small-world' networks," *Nature*, vol. 393, 1998.
- P. Erdős and A. Rényi, "On random graphs I," *Publicationes Mathematicae*, vol. 6, 1959.

Thank You

Questions & Discussion

https://github.com/TAR2003/Feedback_Vertex_Set_Problem_Algorithm_Project_4-2

https://www.kaggle.com/datasets/tawkirazizrahman/fvs-synthetic-dataset-100k

https://www.kaggle.com/datasets/tawkirazizrahman/fvs-synthetic-dataset-20k